

AMANA I LAB

“Build a RAG system with LangChain that answers questions from a PDF. How would you implement it?”

Build RAG From Scratch Using LangChain

PART 1: Theory & Architecture → PART 2: Full Code → PART 3: Interview Follow-Ups

The code working is only half the interview. This guide walks through the theory first, then the full implementation, then the follow-up questions that come after — the same structure a real interview actually follows.

amanailab.com/notes · YouTube @AmanAILab · LinkedIn: aman-chauhan71

PART 1

Theory & Architecture

01

What Is RAG?

Retrieval-Augmented Generation, in plain language

Instead of relying only on what a model memorized during training, RAG looks up relevant information at answer-time and gives it to the model as context. This is what lets a model answer questions about a specific PDF it has never seen before — the answer isn't stored in its weights, it's retrieved and handed to it fresh.

Without RAG

Answers only from training data — can't know about your PDF, and can confidently guess wrong.

With RAG

Retrieves the exact relevant passage from your PDF first, then answers grounded in it.

02

Why Do We Need Retrieval At All?

Three reasons the naive approach breaks down

1

The model never saw your PDF

It was trained on public data long before your document existed — it has zero knowledge of it.

2

Context windows are limited

You can't paste an entire 300-page PDF into every prompt — retrieval finds only the relevant slice.

3

Answers need to be grounded

Retrieval gives the model something concrete to point to, instead of guessing from memory.

03

The Full Architecture

Eight steps, PDF to final answer

Two distinct phases: **ingestion** (done once, when you load your document) and **query time** (runs every time a user asks a question).

■ **PHASE 1 — INGESTION (once):** PDF → Document Loader → Text Splitter → Embeddings → Vector Database

■ **PHASE 2 — QUERY TIME (every question):** Question → Retriever → Prompt → LLM → Answer

04

What Each Piece Actually Does

Document loader, chunking, embeddings, vector store, retriever

Document Loader

Reads the raw PDF and extracts its text into a format LangChain can work with. Each page typically becomes one Document object with the page's text plus metadata (like page number).

Text Splitter (Chunking)

Breaks the document into smaller pieces. `chunk_size` controls how big each piece is; `chunk_overlap` keeps a little repeated text between chunks so an idea isn't cut in half at a boundary.

Embeddings

Converts each chunk into a list of numbers capturing its MEANING. Chunks with similar meaning get similar number patterns — this is what makes semantic search possible.

Vector Store

Indexes every chunk's embedding so it can quickly find the most similar ones to any new query. FAISS is a fast, local, in-memory choice good for a demo — no external service needed.

Retriever

Wraps the vector store into a simple interface: give it a question, get back the top-k most relevant chunks.

PART 2

The Full Implementation

00

Project Structure

Two files: ingestion and query

```
rag-interview/  
  data/document.pdf  
  ingest.py <- loads, splits, embeds, stores  
  rag.py <- retriever, prompt, chain, answer
```

1 Load the Document

Read the raw PDF and extract its text into Documents LangChain can work with.

```
from langchain_community.document_loaders import PyPDFLoader  
  
loader = PyPDFLoader("data/document.pdf")  
documents = loader.load()
```

2 Split Into Chunks

Break the document into overlapping pieces so ideas aren't cut in half at a boundary.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter  
  
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=1000,  
    chunk_overlap=200  
)  
chunks = splitter.split_documents(documents)
```

3 Create Embeddings

Convert each chunk into a meaning-capturing vector, used for semantic search.

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small"
)
```

4 Store in a Vector Database

Index every chunk's embedding for fast similarity search. FAISS is a good local choice for a demo.

```
from langchain_community.vectorstores import FAISS

vectorstore = FAISS.from_documents(chunks, embeddings)
vectorstore.save_local("faiss_index") # persist for rag.py to load
```

5 Create the Retriever

Wrap the vector store into a simple interface: give it a question, get back the top-k relevant chunks.

```
retriever = vectorstore.as_retriever(  
    search_kwargs={"k": 3}  
)
```

6 Build the Prompt

Combine the retrieved context with the user's question into one instruction for the model.

```
from langchain_core.prompts import ChatPromptTemplate  
  
prompt = ChatPromptTemplate.from_template("""  
Answer using ONLY the context below.  
  
Context:  
{context}  
  
Question: {question}  
""")
```

7 Build the RAG Chain

Wire retriever, prompt, and model together into one runnable chain using LCEL.

```
from langchain_openai import ChatOpenAI
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)
```

8 Ask a Question

Invoke the chain — retrieval, prompting, and generation all happen in one call.

```
answer = rag_chain.invoke("What is the main objective of this document?")
print(answer)
```

PART 3

The Interview Discussion

The code working is only half the interview

01

Five Follow-Up Questions

What the interviewer asks right after your code works

01

Why did you use RecursiveCharacterTextSplitter specifically?

Must Know

■ The most common immediate follow-up to any chunking code

STRONG ANSWER

It tries to split on natural boundaries first — paragraphs, then sentences — before falling back to raw characters. This keeps chunks more semantically coherent than a naive fixed-character split, which could cut a sentence in half at an arbitrary point.

■ **Follow-up:** What would you use instead if you were splitting code files rather than prose?

■ **Common mistake:** Saying 'because it's the default in tutorials' — the interviewer wants the actual reasoning, not that it's commonly used.

★ **Pro tip:** Mentioning that LangChain has language-specific splitters (like one for Markdown or code) shows you know this choice isn't one-size-fits-all.

02

How would you choose chunk size and overlap?

Must Know

■ Direct follow-up, tests practical judgment beyond copying default numbers

STRONG ANSWER

Chunk size is a tradeoff: too small and you lose surrounding context; too large and you dilute relevance (a chunk about many topics matches poorly against a specific question) and waste tokens. Overlap should be large enough to preserve an idea that spans a boundary, but not so large that it duplicates most of the chunk. In practice, this is tuned empirically against your actual documents and queries, not picked once from a rule of thumb.

■ **Follow-up:** How would you actually TEST whether your chunk size choice is good, rather than just guessing?

■ **Common mistake:** Giving a single 'magic number' answer without acknowledging it depends on the document type and would be tuned empirically.

★ **Pro tip:** Naming the specific tradeoff (context loss vs. relevance dilution) shows you understand WHY the parameter matters, not just that it exists.

03**Why use a vector database at all — why not just search the text directly?**

Must Know

■ Tests understanding of the actual problem vector search solves

STRONG ANSWER

Plain text search (keyword matching) only finds exact word overlaps — it would miss a relevant passage that answers the question using completely different wording. A vector database enables SEMANTIC search: finding chunks that are conceptually related to the query, not just ones sharing the same words. It's also built for fast similarity search at scale, which a naive loop-and-compare approach isn't.

■ **Follow-up:** At what point would a simple in-memory brute-force search stop being good enough, and what would you switch to?

■ **Common mistake:** Only mentioning speed/scale as the reason — the semantic (meaning-based) matching capability is the more fundamental reason, even before scale becomes a concern.

★ **Pro tip:** Leading with the semantic search argument, then adding scale as a secondary point, orders your answer by importance correctly.

04

What happens if the retriever returns irrelevant documents?

Advanced

■ Tests awareness of a real failure mode, not just the happy path

STRONG ANSWER

The model may either hallucinate an answer by loosely connecting to the irrelevant context, or give a low-quality answer that technically references the context without really answering the question. A production system would check the retrieval SCORE — if the best match isn't actually similar enough (below some threshold), the system should say it doesn't have enough information rather than forcing an answer from weak context.

■ **Follow-up:** How would you pick the right similarity score threshold for that refusal check?

■ **Common mistake:** Assuming the model will 'just know' the context isn't relevant and handle it gracefully — without an explicit check, it often won't.

★ **Pro tip:** Proposing a concrete threshold-based refusal mechanism (not just 'prompt it to be careful') shows you can turn this into an actual implementation, not just a concern.

05

How would you improve retrieval quality beyond this basic version?

Advanced

■ The closing question — tests whether you know what's beyond the demo

STRONG ANSWER — NAME MULTIPLE OPTIONS

- **Hybrid search:** combine keyword (BM25) search with vector search, catching exact terms that semantic search alone can miss.
- **Reranking:** a second, more careful scoring pass on the top candidates from initial retrieval, to reorder by true relevance.
- **Metadata filtering:** narrow the search using structured fields (like department or date) before running similarity search.
- **Query rewriting / multi-query:** generate several reworded versions of the question to retrieve against, then merge results.

■ **Follow-up:** Which of these would you implement FIRST if retrieval quality was the top complaint, and why?

■ **Common mistake:** Naming only one improvement instead of the full toolkit — this question specifically tests breadth of knowledge about what's beyond the basic pipeline.

★ **Pro tip:** Naming all four techniques by name, briefly, shows you've studied production RAG patterns beyond just the tutorial-level implementation.

00

The Night-Before Cheat Sheet

Theory, code steps, and follow-ups compressed

	What is RAG	Retrieval-Augmented Generation: look up relevant info at answer-time, give it to the model as context.
	Why retrieval is needed	Model never saw your PDF; context windows are limited; answers need to be grounded, not guessed.
	The 8-step architecture	PDF-Loader-Splitter-Embeddings-VectorDB (ingest once); Question-Retriever-Prompt-LLM (every query).
	Load document	<code>PyPDFLoader('data/document.pdf').load()</code>
	Split into chunks	<code>RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)</code>
	Create embeddings	<code>OpenAIEmbeddings(model='text-embedding-3-small')</code>
	Store in vector DB	<code>FAISS.from_documents(chunks, embeddings)</code>
	Create retriever	<code>vectorstore.as_retriever(search_kwargs={'k': 3})</code>
	Build prompt	ChatPromptTemplate with {context} and {question} variables
	Build RAG chain	<code>LCEL: {context, question} prompt llm StrOutputParser()</code>
	Ask a question	<code>rag_chain.invoke('your question here')</code>
	Why RecursiveCharacterTextSplitter	Splits on natural boundaries (paragraph/sentence) before raw characters — more coherent chunks.
	Choosing chunk size/overlap	Tradeoff: too small loses context, too large dilutes relevance. Tune empirically.
	Why a vector database	Enables SEMANTIC search (meaning-based), not just exact keyword matching — plus scale.
	Irrelevant retrieved docs	Check similarity score against a threshold; refuse to answer confidently if below it.
	Improving retrieval quality	Hybrid search, reranking, metadata filtering, query rewriting/multi-query.

The interview takeaway: a working RAG pipeline is table stakes. Being able to explain WHY each piece is built the way it is — and what you'd improve beyond the basic version — is what separates a pass from a strong pass.

Download All Free PDFs: amanailab.com/notes AmanAI Lab · amanailab.com · YouTube @AmanAILab ·
LinkedIn: [aman-chauhan71](#) GitHub: [amanailab](#) · Instagram: [@amanailab](#) · aman.chauhan.ai71@gmail.com